

Greedy Algorithms

Reading:

- CLRS,
  - Chapter 17 (1st ed.)
  - Chapter 16 (2nd and 3rd ed.)

Greedy Algorithms

A *greedy algorithm* always makes the choice that looks best at the moment.

– Everyday example: Driving in heavy traffic

– The hope: a locally optimal choice will lead to a globally optimal solution

– For some problems, it works

Activity-Selection Problem



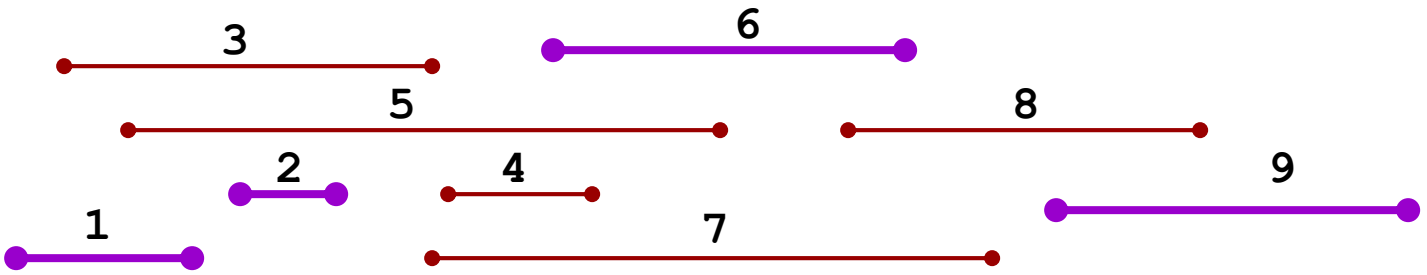
- Problem: get your money's worth out of an amusement park
  - Buy a wristband that lets you onto any ride
  - Lots of rides, each starting and ending at different times
  - Your goal: ride as many rides as possible
    - Another, alternative goal that we don't solve here: maximize time spent on rides
- Welcome to the *activity selection problem*

Activity-Selection

- Formally:
  - Given a set $S = \{a_1, a_2, \ldots, a_n\}$ of n activities

    s_i = start time of activity i

    f_i = finish time of activity i
  - Find max-size subset A of non-conflicting activities



- Assume (w.l.o.g) that $f_1 \leq f_2 \leq \ldots \leq f_n$

Greedy Choice Property

- The activity selection problem exhibits the *greedy-choice property (GCP)*:
  - Locally optimal choice $\Rightarrow$ globally optimal solution
- Theorem: if S is an activity selection instance sorted by finish time, then there exists an optimal solution $A \subseteq S$ such that $a_1 \in A$.
- Proof: Given a solution B that does not contain a_1, replace the first activity in B with a_1. The resulting solution is feasible (why?), it has the same number of activities as B, and it includes a_1.

Activity Selection: A Greedy Algorithm

- So actual algorithm is simple:
  - Sort the activities by finish time
  - Select the first activity
  - Then select the next activity in sorted list which starts after previous activity finishes
  - Repeat until no more activities
- Time complexity: O(n log n)
- Intuition is even more simple:
  - Always pick the earliest to finish ride available at the time.

Optimality of the Greedy Algorithm

The GCP refers to the first step of the algorithm.

In order to prove that the greedy algorithm is optimal, we prove the following claim:

Induction Claim: For every k, after selecting k activities, there exists an optimal solution that includes the first k activities selected by Greedy.

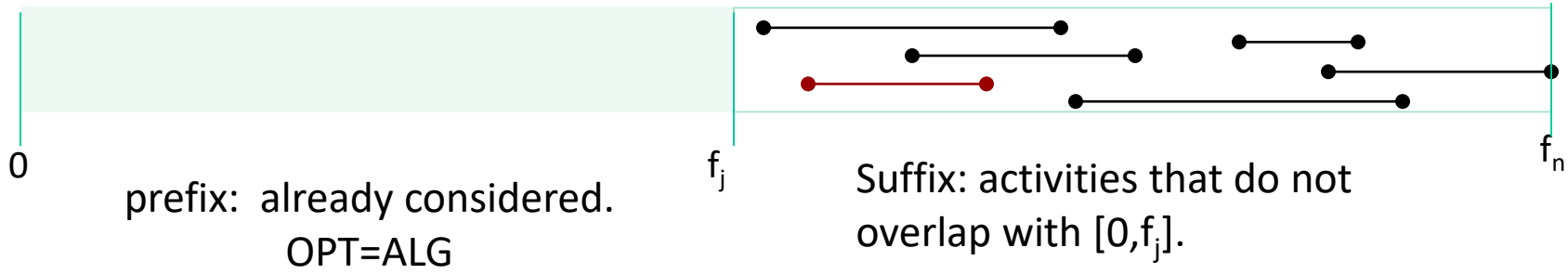
Base case: k=0. An empty solution is a subset of any optimal solution.

Induction step: Let a_j be the k-th activity in the solution of greedy. Induction hypothesis: there exists an optimal solution, OPT, that agrees with greedy up to time f_j (=finish time of activity a_j)

Optimality of the Greedy Algorithm

Induction step (cont'): OPT includes the k activities selected so far + an optimal solution for the suffix.

The prefix and the suffix have no overlap. Therefore, the g.c.p. is exactly what we need for the induction step – it considers an optimal solution for the suffix. The (k+1)st chosen activity is the first activity chosen from of the suffix.



A general structure of a greedy algorithm and proof.

1. preprocessing (like sort)
2. a sequence of local optimal choices.

Optimality proof: step 1. The greedy-choice property

If *S* is an instance whose local optimum is to perform step 'a'
then there exists an optimal solution A that agrees with 'a'

Proof Method: Given a solution B that does not agree with a,
show that either
(i) B is not optimal, or
(ii) [exchange argument] B can be modified to agree with a,
and the resulting solution is feasible and is not worse than A.

A general structure of a greedy algorithm and proof.

Optimality proof: step 2. Induction on the number of steps.

Induction Claim: For every k, there exists an optimal solution that agrees with the first k steps of the greedy algorithm.

Base case: k=0. An empty solution is a subset of the optimal solution.

Induction step: Based on the greedy choice property.

In particular, when the greedy algorithm terminates, there exists an optimal solution that is identical to the solution provided by the greedy algorithm.

Supermarket Sale

- A new sale just started in the local supermarket: every two items you buy together, you get the cheaper one in half the price.
- You want to buy $2n$ products, with prices $p_1, \ldots, p_{2n}$.
- Suggest an efficient algorithm that decides how to order the products in pairs, such that the total cost is minimal.
- Prove the correctness of your algorithm.





Supermarket Sale

Example: Assume $p_1 = 7,\ p_2 = 15,\ p_3 = 5,\ p_4 = 10$

If we pair the products $(1, 2), (3, 4)$ then we will pay

$$15 + 3.5 + 10 + 2.5 = 31.$$

However, if we pair the products $(2, 4), (1, 3),$ then we will pay $15 + 5 + 7 + 2.5 = 29.5$.

Supermarket Sale

Algorithm:

Sort the items $X = \{p_1, p_2, \ldots, p_{2n}\}$ by cost in non-increasing order.

Return $(p_1, p_2), (p_3, p_4), \ldots, (p_{2n-1}, p_{2n})$.

Running time:

Dominated by the sorting, $O(n \cdot \log n)$. The pairing takes $O(n)$.

Supermarket Sale

- Greedy Choice Property: Let x, y be the two highest prices in X. Then there exists an optimal solution containing the pair (x, y).
- Proof: Let S be a solution to the problem, and assume $(x, y) \notin S$. Let $(x, z), (y, w) \in S$.
- The price of these two pairs is $x + \frac{z}{2} + y + \frac{w}{2}$.
- Since $w, z \leq y$, the price of the pairs $(x, y), (z, w)$ is

$$x + \frac{y}{2} + \max[z, w] + \frac{\min[z, w]}{2} \leq x + \frac{z}{2} + y + \frac{w}{2}$$

- Therefore, we can exchange $(x, z), (y, w)$ with $(x, y), (z, w)$ without hurting the total price.

Supermarket Sale

Induction claim: for every $0 \leq k \leq n$ there exists an optimal solution B such that $(p_1, p_2), \dots, (p_{2k-1}, p_{2k}) \in B$.

Proof: For $k = 0$, trivial.

- Let $k > 0$ and let B be an optimal solution such that $(p_1, p_2), \dots, (p_{2k-3}, p_{2k-2}) \in B$.
- Since p_{2k-1}, p_{2k} are the highest remaining prices, from the GCP there exists a solution B' for $p_{2k-1}, \dots, p_{2n}$ such that p_{2k-1}, p_{2k} is in it.
- Therefore, there exists an optimal solution with the pairs $(p_1, p_2), \dots, (p_{2k-1}, p_{2k})$.

Coin Changing

- Goal: Given currency denominations: 1, 5, 10, 25, 100, devise a method to pay amount to customer using fewest number of coins.
- Ex: 34¢.



- Cashier's algorithm: At each iteration, add coin of the largest value that does not take us past the amount to be paid.
- Ex: $2.89.



Coin-changing Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

Coin-Changing: Greedy Algorithm

Cashier's algorithm: At each iteration, add a coin of the largest value that is not higher than the remaining amount to be paid.

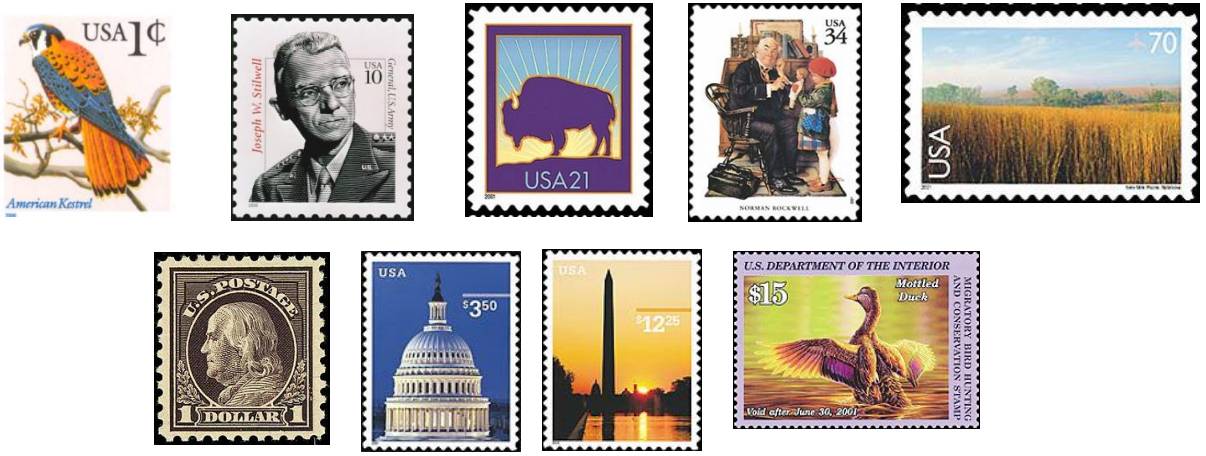
```
Sort coins denominations by value: c1 < c2 < … < cn.
Given input value x:

S ← ϕ        (coins selected)
while (x ≠ 0)  {        (remaining value)
    if (c1 > x)
        return "no solution"
    let k be the largest integer such that ck ≤ x
    x ← x - ck
    S ← S ∪ {k}
}
return S
```

Takes $O(n \log n) + |S|$ where $|S|$ is the number of coins in the solution.

Coin-Changing: Analysis of Greedy

- Note: The greedy algorithm does not solve any set of c_i's
- Example: US postal denominations: 1, 10, 21, 34, 70, 100, 350, 1225, 1500.



- Consider x=140¢.
  - Greedy: 100, 34, 1, 1, 1, 1, 1, 1. Optimal: 70, 70.
- Moreover, if $c_1 > 1$, Greedy may not even lead to feasible solution. Example: coin set C = {7, 8, 9}, x = 15.

Coin-Changing: Analysis of Greedy

Theorem: For any value x, Greedy is optimal for U.S. coinage: 1, 5, 10, 25, 100.

Proof: Warm-up: Constraints on an optimal solution:

k	c_k	All optimal solutions must satisfy
1	1	P < 5
2	5	N <2
3	10	N + D < 3
4	25	Q < 4
5	100	no limit

P = pennies,
N = nickels,
D = dimes,
Q = quarters

For example, a solution with P≥5 is not optimal, because 5 pennies can be replaced by a single nickel – reducing the number of coins by 4.

Coin-Changing: Analysis of Greedy

Greedy choice property:

1. If $c_k \leq x < c_{k+1}$ then the optimal solution includes coin k.
2. if $x \geq c_n$ (=100) then the optimal solution includes coin n.

Proof: Assume that $c_k \leq x < c_{k+1}$, but the solution does not include coin k. Then, it needs enough coins of type c_1, …, c_{k-1} to add up to x, and can't be optimal.

- If Greedy takes a dollar, then any other way to get 100c implies Q>3 or N+D>2 or N>1 or P>4.
- If Greedy takes a quarter, then any other way to get 25c implies N+D>2 or N>1 or P>4.
- If Greedy takes a dime, then any other way to get 10c implies N>1 or P>4.
- If Greedy takes a nickel, then any other way to get 5c implies P>4.

Similarly, if $x \geq c_n$ and coin n is not included.

Coin-Changing: Analysis of Greedy

Given the g.c.p, proceed with the optimality proof by induction on the value of x.

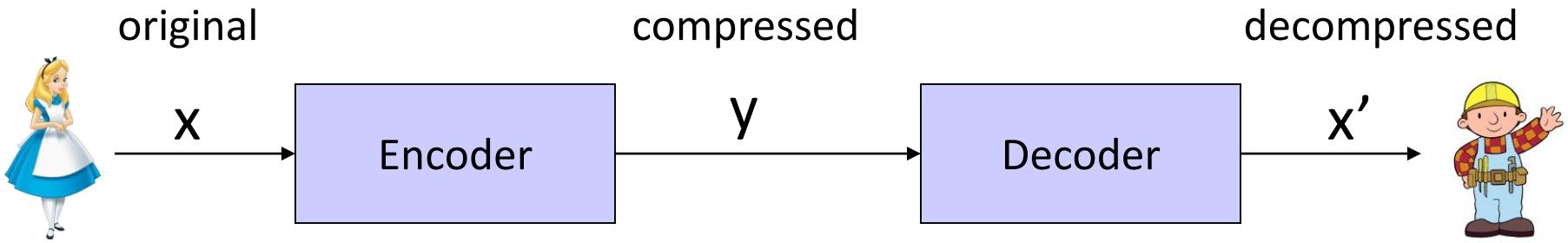
Base case: (i) x=0, no coin is used. (ii) if $0<x<c_1$ – no solution exists and the algorithm reports it correctly.

Step: Assume that greedy is optimal for every value less than x. Let $c_k \leq x < c_{k+1}$. Greedy takes coin k.

By the GCP there exists an optimal solution that takes coin k. Therefore, the problem reduces to coin-changing of the value $x - c_k$ cents, which, by the induction hypothesis, is optimally solved by greedy.

Data Compression

Basic concepts



- Lossless compression $x=x'$
  - Also called entropy coding, reversible coding.
- Lossy compression $x \neq x'$
  - Also called irreversible coding.
- Compression ratio = $|x|/|y|$
  - $|x|$ is number of bits in x.

Why Compress?

- Save storage space
- Reduce time for transmission
  - Faster to encode, send, then decode than to send the original
- Progressive transmission
  - Some compression techniques allow us to send the most important bits first so we can get a low resolution version of some data before getting the high fidelity version
- Reduce computation
  - Use less data to achieve an approximate answer

Lossless Compression

- Data is not lost - the original is really needed.
  - text compression
  - compression of computer binaries to fit on a CD/ DoK.
- Compression ratio typically no better than 4:1 for lossless compression.
- Major techniques include
  - Huffman coding
  - Arithmetic coding
  - Dictionary techniques (Lempel-Ziv 1977)
  - Sequitur (Nevill-Manning, Witten 1996)
  - Standards - Morse code, Braille, Unix compress, gzip, zip, GIF, JBIG

Lossy Compression

- Data is lost, but not too much.
  - audio
  - video
  - still images, medical images, photographs
- Compression ratios of 10:1 often yield quite high fidelity results.
- Major techniques include
  - Vector Quantization
  - Wavelets
  - Transforms
  - Standards - JPEG, MPEG

Fixed-length Code

Suppose we have a data file with 100,000 characters that we wish to store compactly.

The file contains only the characters a-f with the following frequencies:

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
Fixed-len	000	001	010	011	100	101

We are looking for a binary code.

why?

In a fixed-length code, we need 3 bits per character. The total length of the code is 300,000 bits.

Variable-length Codes

Can we reduce the total size by using variable-length code?

We limit ourselves to prefix-free codes: no codeword is a prefix of another codeword.

This guarantees a simple one-pass decoding. Each code string has a unique parse, no need of 'end of character' sign.

Example: Consider the non-prefix-free code

C(a)=0, C(b)=01, C(c) =10.

The string 010 is a decoding of both 'ac' and 'ba'

Huffman Coding (1951)

- Uses frequencies of symbols in a string to build a variable length prefix-free code.
  - Each symbol is mapped to a binary string.
  - More frequent symbols have shorter codes.
  - No code is a prefix of another.

Variable-length Codes

Can we reduce the total size by using variable-length code?

Yes!

In our example:

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
variable-len	0	100	101	111	1101	1100

The total code length is

45k·1 + 13k·3 + 12k·3 + 16k·3 + 9k·4 + 5k·4 =224,000.

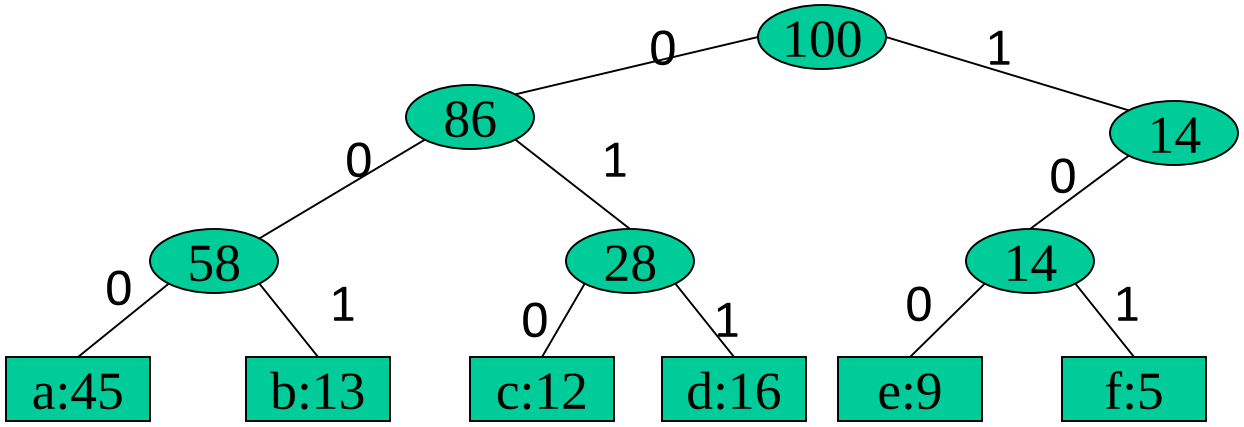
Improving by ~25% the 300,000 we had before.

Tree Representation of a binary-code

Every binary prefix-free code can be represented by a binary tree.

Each left-edge is marked 0, each right-edge is marked 1.

Each leaf represents a character. The path to the leaf determines the decoding of the character.



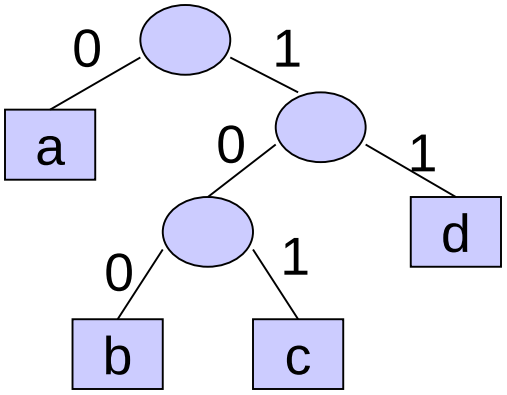
Each internal node is marked with the sum of weights of the leaves in its sub-tree.

Example 1: The tree representing the fixed-length code

Encoding and Decoding

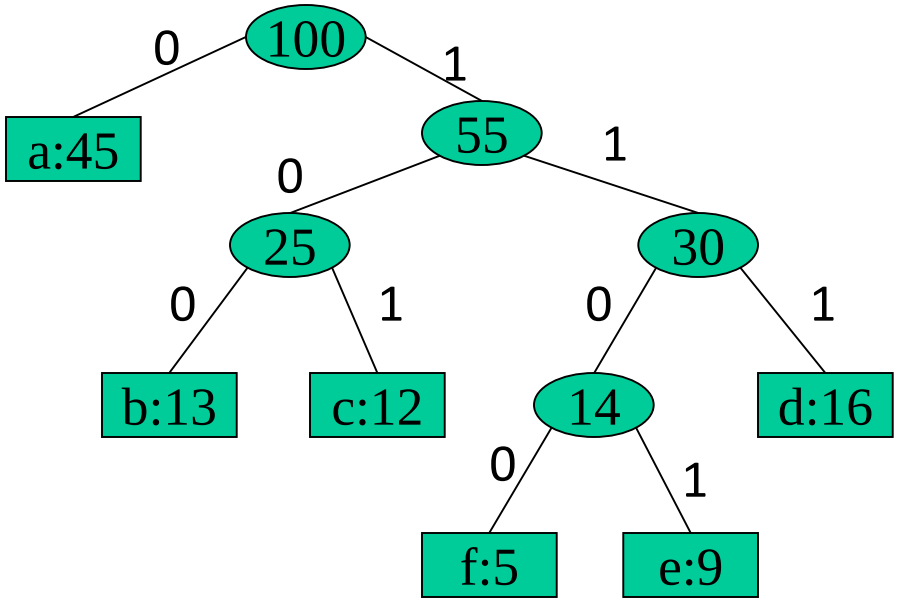
- Encoding: send the code, then the encoded data
  - x = aabddcaa
  - c(x) = a0,b100,c101,d11,00100111110100

- Decoding: Build the tree, then use it to decode.




```
repeat
    start at root of tree
    while current node is not a leaf
        if read bit = 1 then: go right
        else: go left
    report leaf
```

Tree Representation of a Binary-code



Example 2: The tree representing the optimal variable-length code

Given a tree T, For each character $a \in C$:

$p(a)$ – the frequency of a in the file.

$d_T(a)$ – the depth of a in the tree (= length of codeword for a)

The cost of the tree = $B(T) = \Sigma_a\, p(a)d_T(a)$

Constructing an Huffman Code

Huffman code – an optimal prefix-free code constructed by a greedy algorithm.

We build the tree in a bottom-up manner:

Starting with |C| isolated trees, we perform a sequence of |C|-1 merging operations to create the final tree.

Data Structure: A priority queue Q, keyed on f (the frequencies).

The objects in the queue are the trees. Whenever we merge two trees, the frequency of the merged tree is the sum of the frequencies of the merged trees.

Constructing an Huffman Code

Huffman(C)

- Init: |C| isolated vertices. Each vertex is a root of a tree of size 1. Each vertex gets a value $p(v_i)$= the frequency of the ith character.
- Repeat |C|-1 times:

  Add a new vertex v, whose children are the two tree-roots u_1,u_2 with minimal p value.

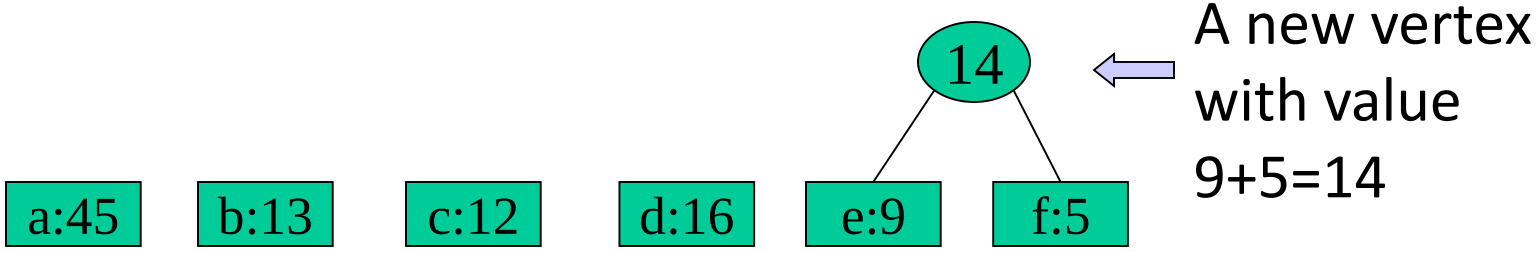
  Determine $p(v)=p(u_1)+p(u_2)$

Constructing an Huffman Code -example

- Init: |C| isolated vertices. Each vertex is a root of a tree of size 1. Each vertex gets a value $p(v_i)$= the frequency of the ith character.

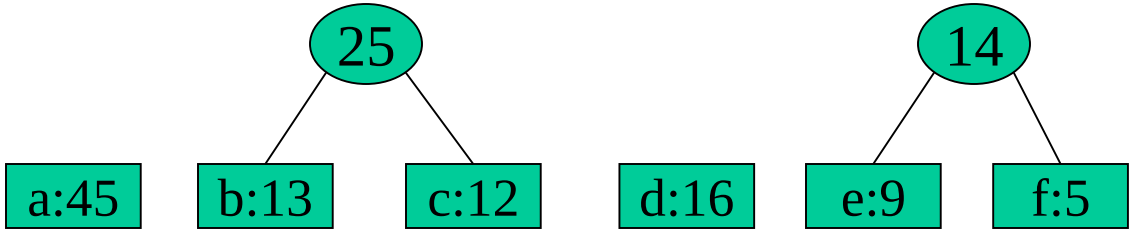
a:45 b:13 c:12 d:16 e:9 f:5

- Step 1: e(9) and f(5) have minimal value.



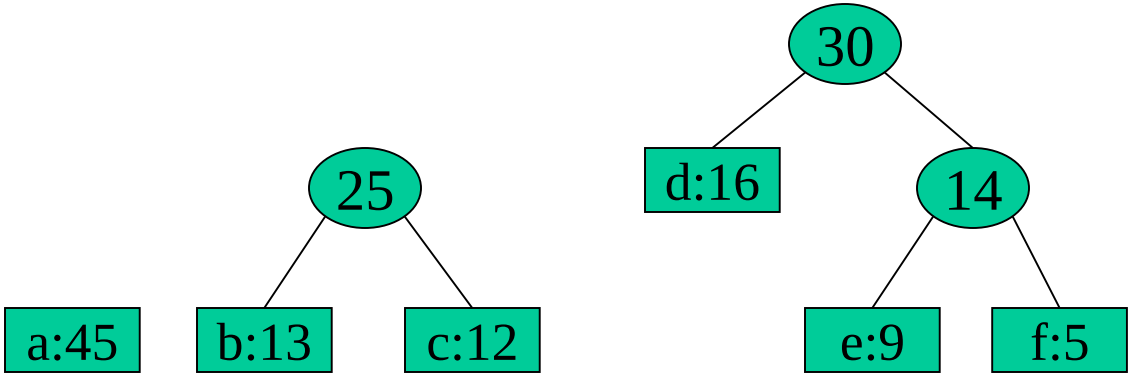
Constructing an Huffman Code –example (2)

- Step 2: b(13) and c(12) have minimal value.



A new vertex with value 12+13=25

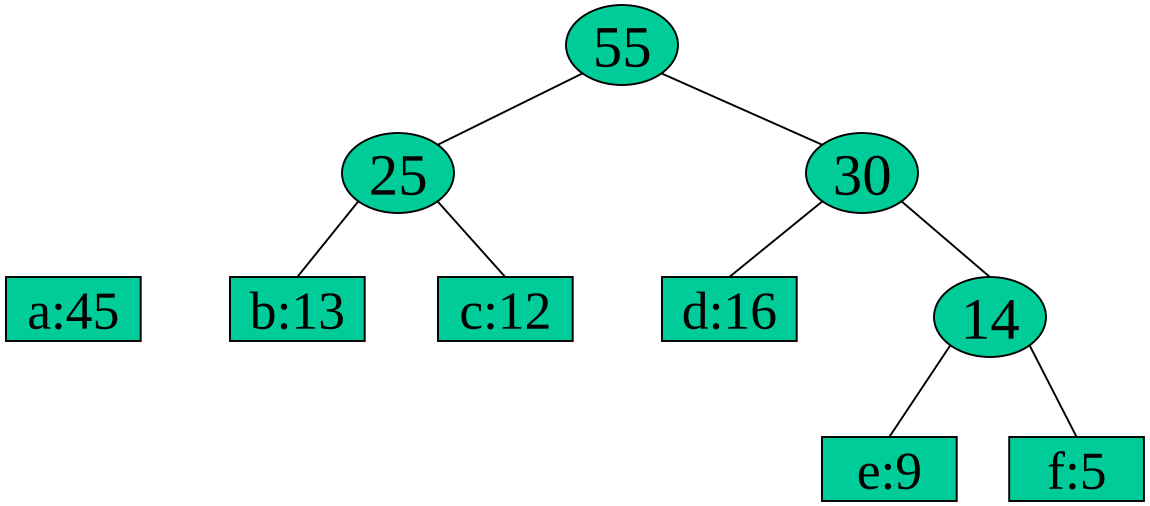
- Step 3: d(16) and ’14’ have minimal value.



A new vertex with value 14+16=30

Constructing an Huffman Code –example (3)

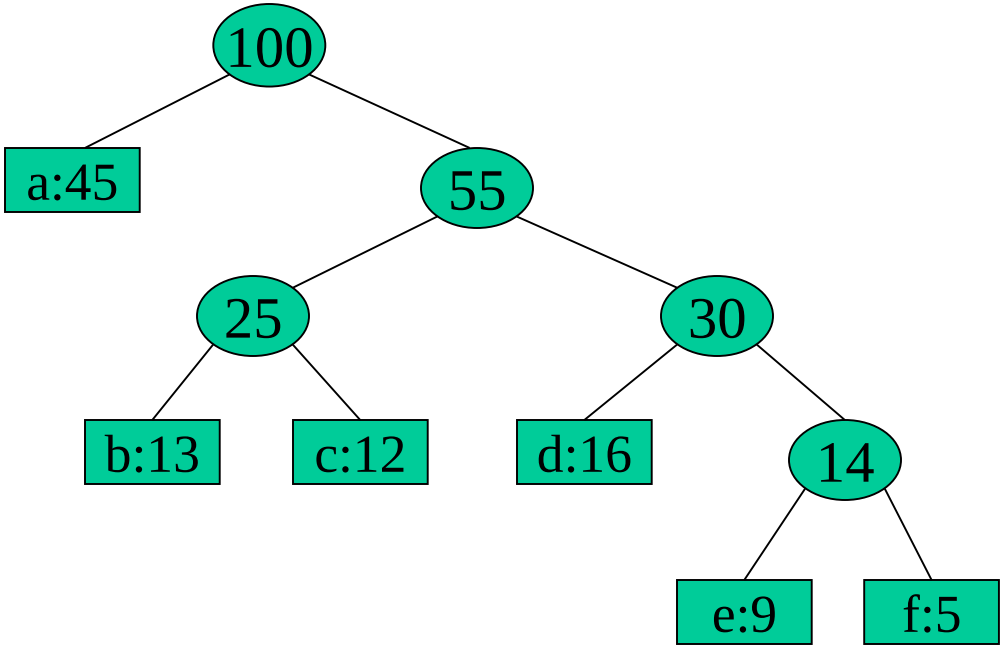
- Step 4: ’25’ and ’30’ have minimal value.



A new vertex with value 25+30=55

Constructing an Huffman Code –example (4)

- Step 5: a(45) and ’55’ have minimal value.

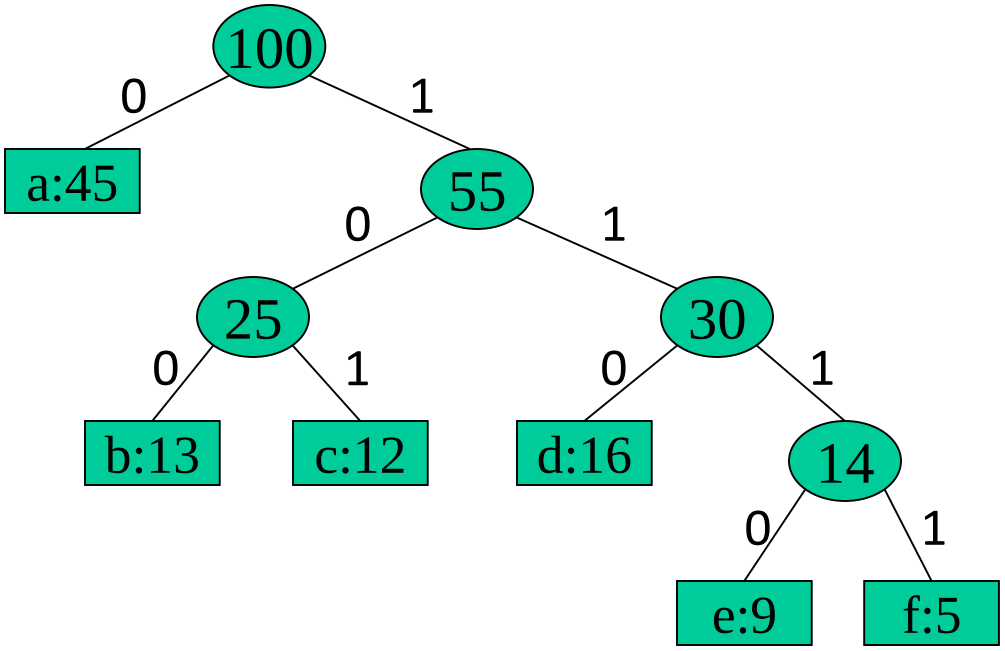


A new vertex with value 45+55=100

Constructing an Huffman Code –example (5)

All the characters are connected.

Label edges with o and 1 to create the code words.

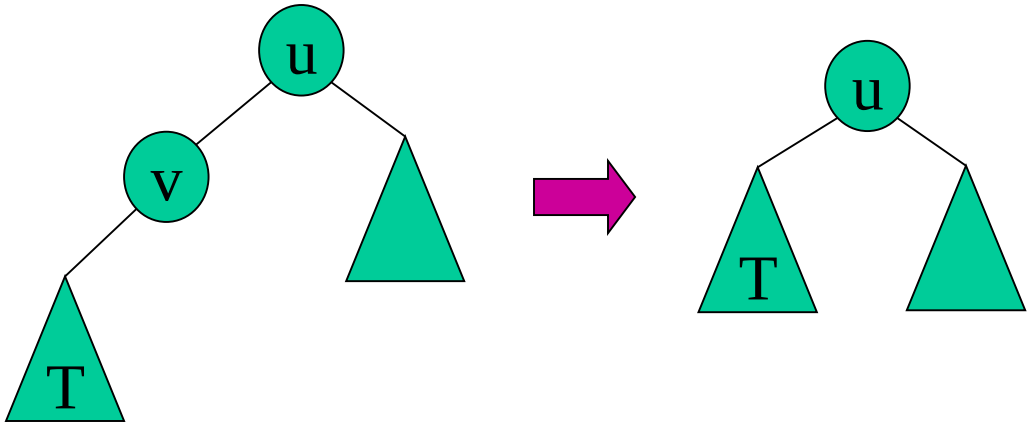


a → 0
b → 100
c → 101
d → 110
e → 1110
f → 1111

Optimality Principle

A simple observation: An optimal code is always represented by a full binary tree (each internal node has two children).

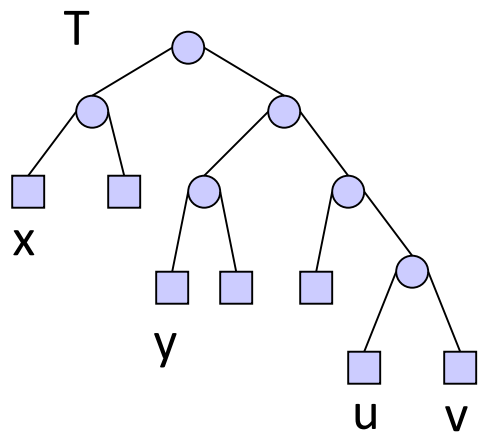
Proof: If we have a node with a singe child, we can improve the coding:



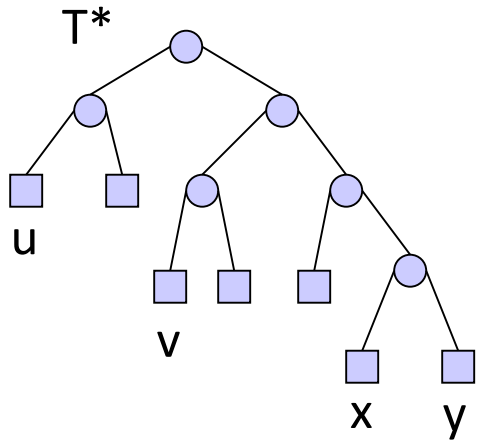
So we know the optimal tree has two children in maximal depth

Optimality of Huffman Code

- Let x,y be the two characters with minimal frequency in the text
- GCP: There exists an optimal tree for C, in which x,y are siblings in maximal depth.
- Proof idea (complete proof in next slide): If not, we can move them to be siblings in maximal depth without hurting the cost.



x smallest
y 2nd smallest
$p(x) \leq p(u)$
$p(y) \leq p(v)$



Optimality of Huffman Code

GCP: There exists an optimal tree for C, in which x,y (minimal frequencies) are siblings in maximal depth.

Proof: Let T be any coding tree, let u,v be siblings in maximal depth and assume w.l.o.g that $p(u) \le p(v)$ and $p(x) \le p(y)$. Replace the codings of u and x and the codings of v and y. Let T^* be the resulting tree.

$$B(T) - B(T^*) = \Sigma_{c\in C}\, p(c)\, d_T(c) - \Sigma c\in C\, p(c)\, d_{T^*}(c)$$

$$= p(x)\, d_T(x) + p(u)\, d_T(u) - p(x)\, d_{T^*}(x) - p(u)\, d_{T^*}(u) +$$
$$p(y)\, d_T(y) + p(v)\, d_T(v) - p(y)\, d_{T^*}(y) - p(v)\, d_{T^*}(v)$$

$$= p(x)\, d_T(x) + p(u)\, d_T(u) - p(x)\, d_T(u) - p(u)\, d_T(x) +$$
$$p(y)\, d_T(y) + p(v)\, d_T(v) - p(y)\, d_T(v) - p(v)\, d_T(y)$$

$$= (p(u)-p(x))\cdot(d_T(u)-d_T(x)) + (p(v)-p(y))\cdot(d_T(v)-d_T(y)) \ge 0$$

Optimality of Huffman Code

We show that Huffman code is optimal by an induction, based on the above greedy choice property.

The proof is by Induction on |C|.
Base case: |C|=2, each character is getting a single bit code – which is clearly optimal.

Optimality of Huffman Code

Step: Assume the algorithm is optimal for an alphabet of $|C|-1$ characters.

Given an instance with $|C|$ characters, after one iteration, we are left with alphabet $H = C \setminus \{x,y\} \cup \{z\}$, where x,y are the characters with minimal frequency in the text and z is a new character such that $p(z) = p(x) + p(y)$.

By the induction hypothesis, the algorithm builds an optimal coding tree T' for H. The output of the algorithm for C is a tree T produced from T' by adding x,y as children of z.

Optimality of Huffman's Algorithm

- By the optimality principles, Huffman algorithm is an optimal greedy algorithm: In each stage, the tree that is built by Huffman is a subtree of some optimal tree.

- In particular, the final tree is optimal.